

작품 제목: 스마트 자세 교정 의자

무게 센서, ToF 센서 그리고 기울기 센서를 이용한 자세 교정 의자

글 | 경성대학교 송승진, 권혁준, 김호건, 이강민, 이동혁

1. 작품 개요

1.1. 개발 배경 및 필요성

현대 사회에서 데스크 업무나 학생들이 공부를 위해 의자에 장시간 앉아 있는 경우가 많다. 이때 잘못된 자세로 인한 거북목 증후군, 척추측만증 등의 근골격계 질환을 유발할 수 있다. 기존 자세 교정 보조 기구들은 대부분 물리적인 지지 기능에만 국한되어 있어, 사용자가 스스로 자신의 나쁜 습관을 인지하고 능동적으로 교정하도록 유도하는 데 한계가 있다. 본 작품은 이러한 문제점을 해결하기 위해 센서 네트워크와 엣지 컴퓨팅(Edge Computing)을 결합하여, 사용자의 자세를 실시간으로 모니터링하고 즉각적인 피드백을 제공하는 '스마트 자세 교정 의자'를 제안한다.

1.2. 작품 목표

본 시스템은 의자의 시트와 등받이에 로드셀(Loadcell)과 비행시간 거리 센서(Time-of-Flight, 이하 ToF)를 부착해 사용자의 하중 분포와 신체 위치 데이터를 수집한다. 센서에서 수집된 데이터를 통해 사용자가 다리를 꼬거나, 한쪽으로 기울어 앉는 비대칭 자세, 거북목 그리고 굽은 자세 등을 제어보드에서 알고리즘을 통해 분석, 판단하게 된다. 사용자는 모바일 앱으로 외부의 도움 없이 자신의 자세를 직관적으로 인지하고 올바른 상태로 수정할 수 있는 환경을 제공받는 것을 목표로 한다.

2. 작품 설명

2.1. 주요 동작 및 특징

본 작품은 STM32[1] 기반 센서 노드에서 수집한 로드셀, ToF 센서 그리고 기울기 센

서의 데이터를 라즈베리파이 5(Raspberry Pi 5, 이하 RPi5)[2]가 실시간으로 수신하여 사용자의 자세를 분석하고, 그 결과를 모바일 앱에 제공하는 엣지 컴퓨팅 기반 자세 교정 시스템이다.

먼저, 각각의 로드셀 채널은 개별적인 보정 계수(factor)를 적용하여 실제 물리량에 근접한 값으로 변환된다. 이때 나온 값과 ToF 센서의 값을 융합하여 안정적인 자세 분석을 가능하도록 하였다. 구체적으로, 로드셀의 좌판·등판 하중 분포와 ToF의 척추 굽힘 거리값을 각각 독립적인 특징(feature)으로 추출한 뒤, 규칙 기반(rule-based) 판단 로직에서 두 센서군의 특징값을 조건부로 결합하여 자세 상태를 종합 판별하는 방식으로 동작한다.

보정된 센서 데이터는 특징 추출 과정을 통해 자세를 설명할 수 있는 지표로 변환된다. 이러한 특징은 규칙 기반으로 동작하며 사용자의 자세를 분석하고 상태를 평가하는 데 활용된다.

자세 판별 결과는 시간에 따라 누적되어 자세 점수로 환산되며, 사용자가 나쁜 자세를 유지할수록 점수가 점진적으로 감소하고, 반대로 바른 자세를 유지하게 되면 점수를 증가하도록 설계하였다.

측정 종료 후에는 대규모 언어 모델(Large Language Model, 이하 LLM) 기반 리포트 생성 모듈이 동작하여, 전체 측정 데이터 기반으로 사용자의 자세 패턴을 분석하고 자연어 형태의 피드백을 생성한다. 해당 리포트는 단순 통계 요약이 아닌, 자세 문제 원인 설명, 시간 흐름에 따른 변화 분석, 맞춤형 운동 추천을 포함하는 형태로 구성되며, 사용자에게 직관적이고 이해하기 쉬운 피드백을 제공한다.

2.2. 전체 시스템 구성

본 시스템은 STM32, RPi5 그리고 모바일 앱으로 구성된 3계층 구조를 갖는다.

STM32는 로드셀, ToF 센서 그리고 기울기 센서로부터 데이터를 수집하여 범용 비동기 직렬 통신(Universal Asynchronous Receiver-Transmitter, 이하 UART)으로

바이너리 패킷 형태로 RPi5에 전송한다. RPi5는 수신된 데이터를 기반으로 센서 보정, 특징 추출, 자세 분류, 점수 계산, LLM 기반 리포트 생성을 수행한다. 모바일 앱은 RPi5와 통신하여 실시간 자세 상태, 센서 분포 시각화 그리고 세션 결과 리포트를 사용자에게 제공한다. 이와 같은 구조를 통해 저전력 STM32는 데이터 수집에 집중하고, 연산 및 분석은 RPi5에서 수행하도록 역할을 분리하여 시스템 효율성과 확장성을 확보하였다. 전체 아키텍처 블록 다이어그램은 그림 1과 같다.

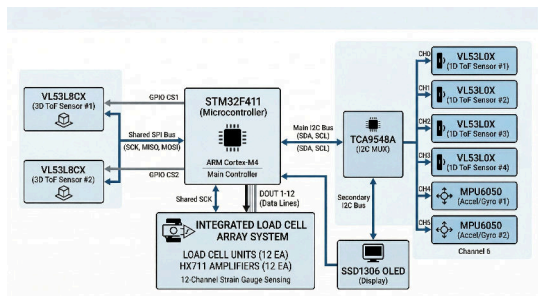


그림 1. 아키텍처 블록 다이어그램

2.2.1. 하드웨어 구성

의자 좌판과 등판에 각 8개씩 총 16개의 로드셀을 배치하고, 등판 중앙의 척추 부위에 1D ToF 센서 4개를 설치하여 굽힘 정도를 측정하도록 하였다. 헤드레스트에는 각각 기울기 센서 그리고 3D ToF 센서를 2개씩 배치하였다. 이러한 센서들의 데이터를 STM32에서 수집하여 RPi5로 전송하는 회로를 설계하였으며, 완성된 의자 사진은 그림 2와 같다.



그림 2. 완성된 의자 사진

2.2.2. 소프트웨어 구성

소프트웨어는 STM32 펌웨어, RPi5 분석 서버, Flutter[3] 기반 모바일 앱으로 구성된다.

STM32 펌웨어는 STM32CubeIDE[4] 및 STM32CubeMX[5] 환경에서 개발되었으며, 센서 초기화, 실시간 데이터 수집, 패킷 생성 그리고 UART 통신을 담당한다. RPi5는 수신된 데이터를 기반으로 센서 보정, 특징 추출, 자세 분류, 점수 계산, 세션 관리, 데이터 저장 그리고 리포트 생성을 수행한다. 모바일 앱은 RPi5 서버와 통신하여 실시간 자세 상태 및 리포트를 사용자에게 시각적으로 제공한다. 특히 RPi5 측 소프트웨어는 규칙 기반 분석 구조와 LLM 기반 리포트 생성 구조를 동시에 지원하도록 설계하였다. 시스템의 전체 시퀀스 다이어그램은 그림 3와 같다.

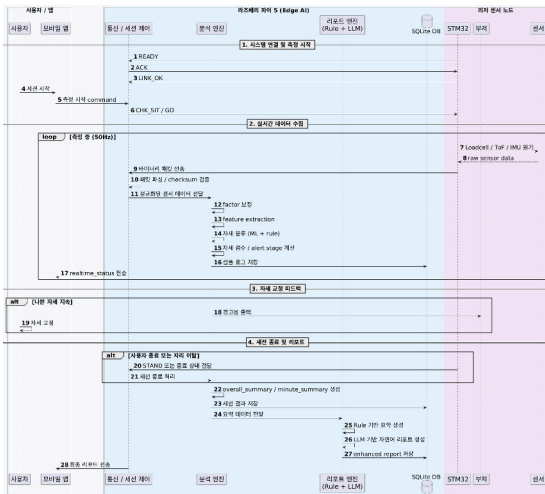


그림 3. 시퀀스 다이어그램

2.3. 개발 환경

본 시스템의 개발에는 소프트웨어 측면에서 C, Python, Dart[6] 언어와 Flutter[3] 프레임워크를 사용하였으며, 하드웨어 측면에서 STM32[1]와 RPi5[2]를 채택하였다. 상세 개발 환경은 표 1과 같다.

구분	개발툴	세부내용
소프트웨어	C	센서 제어
	Python	RPi5 서버
	dart	모바일 앱
	flutter	앱 제작
	회로도 설계	회로 모델링
하드웨어	Edge Device	연산 및 분석
	MCU	센서 값 처리

표 1. 개발환경

3. 단계별 제작과정

3.1. 하드웨어 제작

본 설계의 전체 회로도는 그림 4와 같다. 크게 제어부, 의사 좌판, 등판, 헤드레스트로 나눌 수 있다.

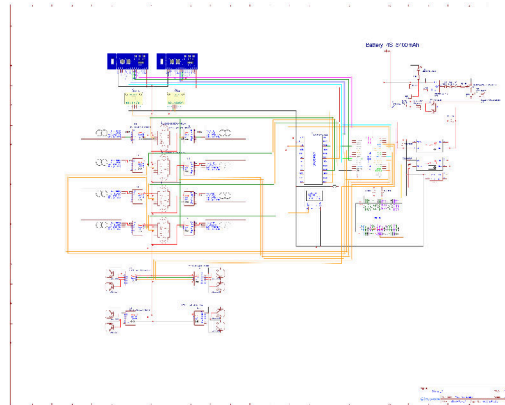


그림 4. 전체 회로도

3.1.1. 제어부

제어부는 STM32, RPi5, 멀티플렉서 (Multiplexer), 전원 회로로 구성된다.

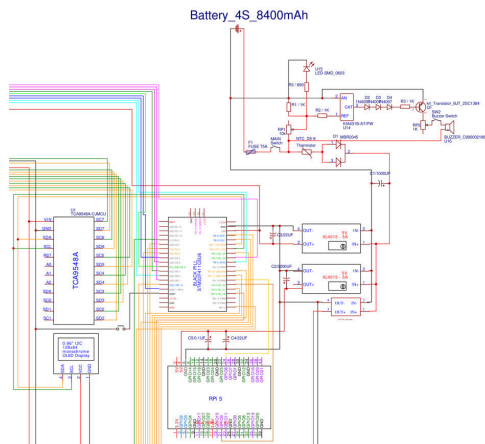


그림 5. 제어부 회로도

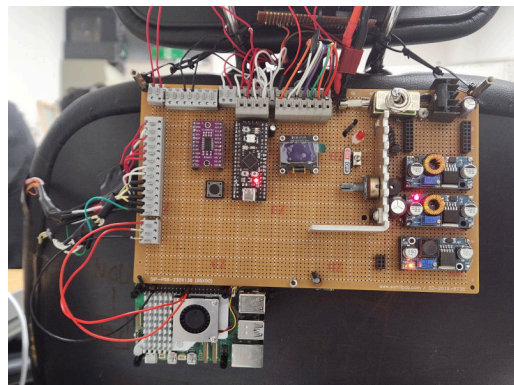


그림 6. 제어부 사진

센서들의 값을 처리하는 주회로와 전원을 공

급하는 공급부, 보호 회로 등은 만능기판 (230 × 140 mm²)에서 제작되었다. 상단의 중심부에서 공급받은 전원을 벅 컨버터의 입력단으로 전달하고, 각 모듈에 필요한 전압으로 변환하여 각 단에 공급한다.

벅 컨버터의 스위칭 노드에서 발생하는 방사 노이즈(Radiated Noise)로 인해 전자기 간섭에 의한 MCU 영향을 감소시키기 위해 다음의 3가지 물리적 감쇄 모델을 사용하여 벅 컨버터와 MCU 간의 적합한 거리를 도출하였다.

먼저, 거리의 역제곱 법칙(Inverse Square Law)에 따라 방사 에너지 밀도는 수식 1과 같이 거리의 제곱에 반비례하여 감소한다.

$$S = \frac{P}{4\pi d^2} PROP \frac{1}{d^2} \dots\dots\dots (1)$$

여기서 S는 단위 면적당 방사 에너지 밀도, P는 노이즈 소스의 방사 전력, d는 소스로부터의 거리이다.

또한, 두 지점 간의 전계 강도 변화는 데시벨 감쇄식으로 수식 2와 같이 표현된다.

$$A_{dB} = 20\log_{10}\left(\frac{d_1}{d_2}\right) \dots\dots\dots (2)$$

여기서 A_{dB}는 데시벨 단위의 감쇄량, d₁은 기준 거리, d₂는 비교 거리이다.

아울러, 자기장 노이즈는 거리의 세제곱에 반비례하여 감소하며 수식 3과 같다.

$$H PROP \frac{1}{d^3} \dots\dots\dots (3)$$

H는 자기장 간섭 강도, d는 소스로부터의 거리이다.

분석 항목	1cm	7cm	결론
방사 노이즈 에너지 밀도	100 [%]	2.04 [%]	약 49배 감소
전계 강도[dB]	0 [dB]	-16.9 [dB]	신호 진폭 85 [%] 감소
자기장 간섭 강도	100 [%]	0.291 [%]	약 343배 감소

표 2. 거리별 방사 노이즈 결과

3가지의 식을 고려하여 MCU와 벅 컨버터의 이격 거리를 7 cm으로 결정하였으며, 추가적인 차폐를 위해 금속 방열판을 설치하여 접지함으로써 전계 노이즈에 대한 차폐 효율(Shielding Effectiveness)을 보강하여 노이즈 마진을 확보하였다.

RPi5 단의 벅 컨버터 출력 시 리플 제거와 저임피던스 경로 제공을 위하여 출력단 바로 앞에 2,200 μF의 대용량 콘덴서를 병렬 설치하였으며, 전원 OFF 시 콘덴서의 잔류 전하를 방전하기 위해 890 Ω 블리더 저항과 LED를 배치하여 잔류 전하 방전을 유도하고, LED의 빛으로 남은 전하를 시각적으로 확인할 수 있다.

STM32F411CEU6[1][7] 단의 XL4005 벅 컨버터 출력 시 저주파 노이즈 및 리플 제거를 위해 STM32F411CEU6의 입력단에 0.1 μF 과 22 μF의 콘덴서를 병렬 설치하였다.

MCU와 멀티플렉서의 이격 거리는 안정적인 I2C 통신 속도를 위해 가깝게 배치하였다. 센서의 전원 공급선 및 통신선의 점퍼는 간편한 장탈착을 위해 푸시 타입 터미널 블록을 채택했다.

시스템의 실시간성을 보장하기 위해 ARM Cortex-M4 코어 기반의 STM32F411CEU6 모델을 메인 MCU로 사용했다[1][8]. 선정 사유는 표 3과 같다.

선정 사유	비고
실시간 연산성능	100MHz의 동작 주파수와 하드웨어 부동 소수점 연산 장치(FPU)를 탑재하여, MPU6050의 복잡한 쿼터니언 연산 및 ToF 센서의 거리 데이터를 지연없이 처리한다.
다중 통신 인터페이스	다수의 I2C 및 SPI 채널을 지원하여 확장성을 확보하였으며, Direct Memory Access 기능을 활용하여 CPU 부하를 최소화하면서 대량의 센서 데이터를 RPi 5로 전송한다.
전력 효율성	전력 소모 대비 연산 능력이 우수하여 배터리 기반 모바일 시스템에 최적화된 제어 환경을 제공한다.

표 3. STM32 선정 사유

엣지 디바이스로서 STM32가 수집한 데이터를 처리하기 위해 RPi5를 사용했다[2]. 선정 사유는 표 4와 같다.

선정사유	비고
엣지 컴퓨팅 & LLM	STM32가 전송한 데이터를 분석하여 LLM 기반의 지능형 자세분석 및 피드백 생성 알고리즘을 실시간으로 구동한다.
데이터 융합	수집된 센서의 데이터값과 신호를 결합하는 센서 퓨전 알고리즘을 수행하며, 사용자 상태에 대한 종합적인 의사결정 노드 역할을 수행한다.
사용자 서비스	분석된 결과를 전용 앱과 연동하여 실시간 모니터링을 지원한다.

표 4. RPi5 선정 이유

다수의 센서를 효율적으로 관리하기 위해

I2C 멀티플렉서인 TCA9548A를 사용했다 [9]. 선정 사유는 표 5와 같다.

선정 사유	비고
주소 충돌 해결	기본 I2C 주소가 고정된 VL53L0X(ToF) 센서 4개와 MPU6050 2개의 주소 충돌 문제를 하드웨어적으로 완벽히 격리하여 해결하였다.
채널 분산	각 채널을 독립적으로 활성화 함으로써 전체 I2C 버스의 용량을 분산시켜, 통신 속도를 높이고 신호 왜곡(Signal Integrity)을 최소화하였다.

표 5. TCA9548A 선정 이유

배터리 전원을 안전하게 이용하기 위해 전원 회로를 설계하여 사용하였다. 회로에 사용된 소자들의 명칭과 설명은 표 6과 같다.

명칭	비고
Battery	시스템의 주 전력원 (4S 8400mAh) 이다.
퓨즈 & 스위치	과전류 차단 및 물리적 전원 제어를 수행한다.
서미스터	초기 전원 투입 시 발생하는 돌입전류(Inrush Current)를 억제한다.
쇼트키 다이오드	SBL-3040PT를 사용하여 역전압 유입을 방지하여 전압강하를 최소화한다.
콘덴서	1000[uF]의 고용량 전해 콘덴서를 배치하여 전압 리플을 제거하고, 버퍼 역할을 수행한다

표 6. 전원회로 소자 명칭 및 설명

배터리 방전 시 전압이 낮아짐을 대비하여, 보호 회로와 병렬로 방전 알림 시스템을 구성하였다. 사용자는 알림을 통해 배터리의 충전 시기를 알 수 있다.

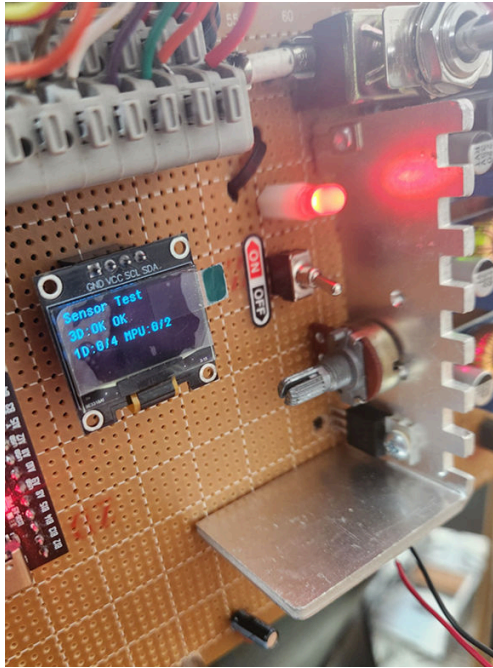


그림 7. LED가 켜져 있는 사진

배터리는 T자형으로 구성된 33 kΩ, 가변저항 10 kΩ 그리고 10 kΩ 저항을 거쳐 선트 레귤레이터 KIA 431B가 인지할 수 있는 전압으로 감쇄한 뒤, 전원 회로 전압이 2.5 V 이하로 낮아질 시 NPN 트랜지스터인 C1384의 베이스(Base)로 전압을 보내고, C1384는 들어온 전압을 부저와 LED를 켜는 데 사용한다.

부저의 소리는 스위치로 ON/OFF가 가능하며, 부저의 스위치와 연결된 가변저항으로 부저의 소리를 조절할 수 있다. T자형에 포함된 가변저항을 조절하여 배터리 종류가 바뀌어도 부저 전압의 기준을 설정할 수 있다.

3.1.2. 의자 좌판

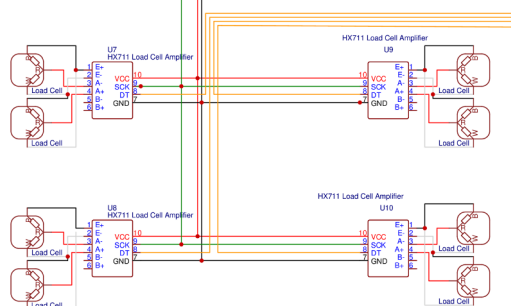


그림 8. 의자 좌판 회로도

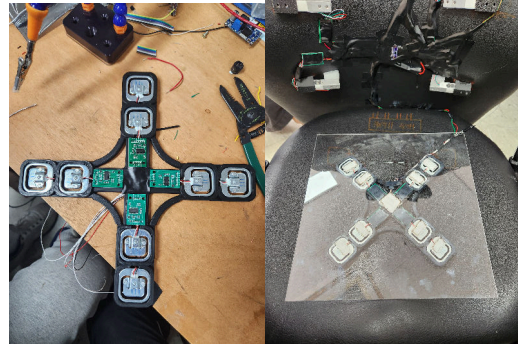


그림 9. 의자 좌판 사진

좌판 하중은 총 4구역으로 나뉘어 측정된다. 좌판 하중이 걸리는 센서 구성은 3선식 로드셀 50 kg 8개 그리고 HX711[10] 4개로 구성된다.

3선식 로드셀 2개를 풀 브리지로 구성하였으며, 로드셀에서 발생하는 미세 아날로그 신호는 HX711 모듈로 전달되는 전선의 선로 저항에도 취약하므로 로드셀과 HX711 사이의 이격 거리를 최소화하여 설치하였다. HX711 모듈에 아날로그 신호를 전송한 후, 변환된 디지털 신호는 STM32에 전송된다.

브라켓은 3D 프린팅을 이용하여 제작하였다. 좌판용 브라켓은 십자(+)형 구조로 설계하여, 중앙 허브에서 4방향으로 팔이 뻗어나가는 형태이다. 각 팔의 끝단에는 타원형 관통 슬롯이 2개씩 배치되어 3선식 로드셀의 체결 홀에 대응하며, 로드셀 2개를 풀 브리지 구성으로 안착시킬 수 있다. 십자형 구조를 채택함으로써 하중이 중앙 허브를 기준으로 4방향에 균등하게 분산되도록 하였다.

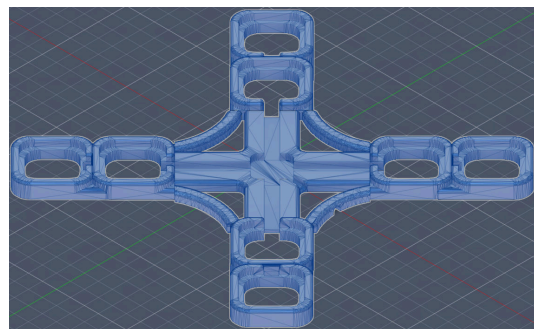


그림 10. 3선식 로드셀 브라켓 3D 모델

3.1.3. 의자 등판

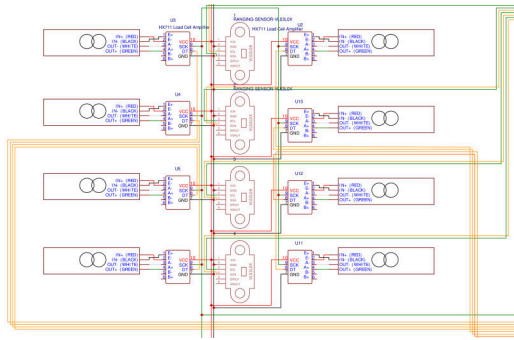


그림 11. 의자 등판 회로도

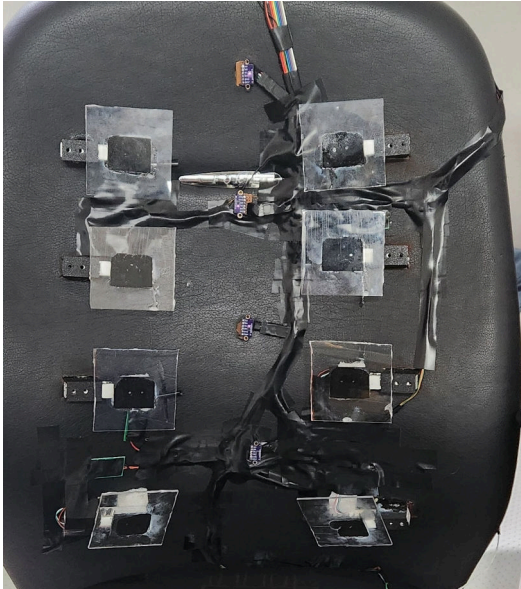


그림 12. 의자 등판 사진

등판 하중의 로드셀은 총 8구역으로 나뉘어 측정된다. 등판 하중이 걸리는 센서 구성은 4선식 막대형 로드셀 20kg 8개 그리고 HX711 8개로 구성된다.

좌판부의 로드셀과 동일하게 4선식 막대형 로드셀과 HX711 사이의 이격 거리를 최소화하여 설치하였다. HX711 모듈에 아날로그 신호를 전송한 후, 변환된 디지털 신호는 STM32로 전송된다. 브라켓은 3D 프린팅으로 제작하였으며, 전방 브라켓과 후방 브라켓 2종으로 구성된다. 전방 브라켓은 평판 베이스 위에 다수의 수직 핀이 병렬로 배열된 빗살 구조로, 등판 메쉬 프레임의 격자

사이에 핀이 끼워져 체결되도록 설계하였다. 후방 브라켓은 두 장의 평행한 벽면과 바닥 판으로 이루어진 U자형 채널 구조이며, 내부에 삼각형 보강 리브를 두어 구조적 강성을 확보하였다. 4선식 막대형 로드셀은 후방 브라켓의 채널 내부에 안착되며, 전방 브라켓이 등판 프레임으로부터 전달되는 하중을 로드셀에 집중시키는 역할을 수행한다.

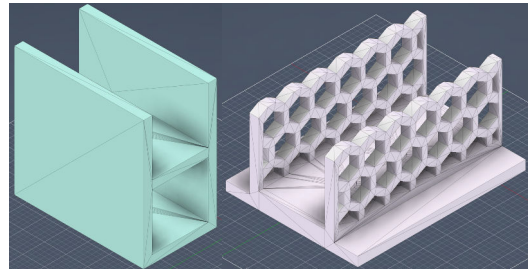


그림 13. 4선식 로드셀 후방 브라켓(좌), 후방 브라켓(우) 3D 모델

등판의 척추 부분에는 1D ToF(VL53L0X)[11] 센서 4개를 장착했다. SDA, SCL 총 2개의 통신선은 I2C 멀티플렉서로 전송되며, I2C 멀티플렉서 채널은 CH2-CH5를 점유한다.

3.1.4. 의자 헤드레스트

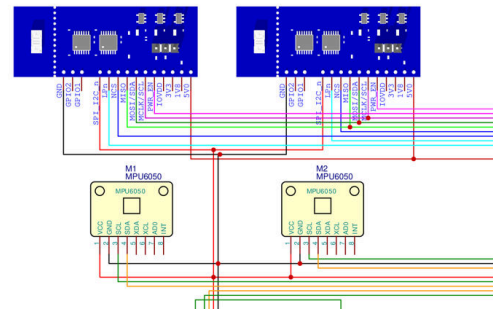


그림 14. 의자 헤드레스트 회로도

헤드레스트의 센서는 기울기 센서인 MPU6050[12] 2개와 3D ToF(VL53L8CX)[13] 2개로 구성된다.

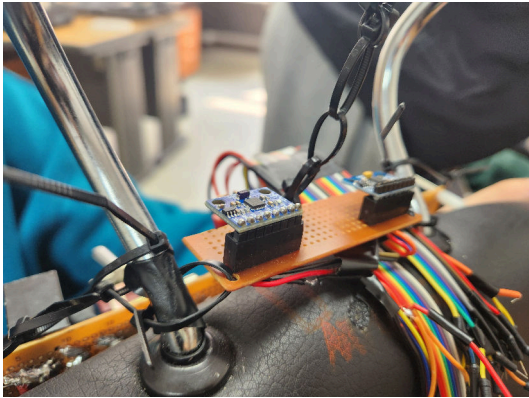


그림 15. MPU 사진

MPU6050은 1D ToF와 동일하게 SDA, SCL 2개의 통신선이 I2C 멀티플렉서로 전송된다. I2C 멀티플렉서의 채널은 CH6-CH7을 점유한다. 지표면과 수평으로 설치하여 의자의 기울기 감도를 측정한다.

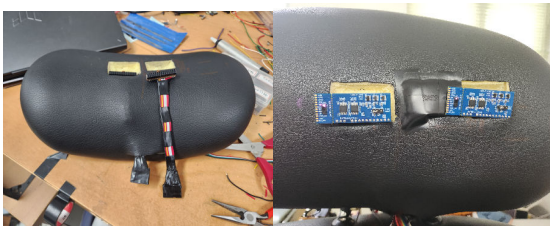


그림 16. 3D ToF 연결부 사진(좌), 설치된 3D ToF(우)

3D ToF는 헤드레스트 중심부에 센서 코어 기준 10 cm 간격으로 나란히 배치하였다 [14]. 낮은 데이터 지연율을 위하여 SPI 통신을 채택하였다. SPI 통신에 사용한 핀은 5V0, GND와 통신 모드를 결정하는 SPI_I2C_N, LPn, NCS, MISO, MOSI, MCLK, PWR_EN 핀 등을 사용하였다. 센서의 공간 해상도 배열(Array)은 4×4(16 Zones) 모드로 설정하여 데이터를 수집하여 STM32로 전송한다.

3.2. STM32 펌웨어 개발

3.2.1. 전체 동작 흐름

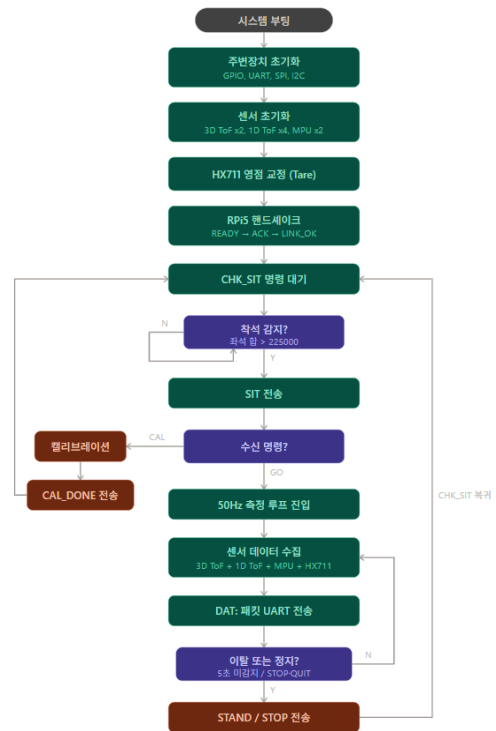


그림 17. STM32 전체 플로우차트

STM32 펌웨어의 전체 동작 흐름은 그림 17와 같다. 전원 인가 시 4종 20개 센서를 순차적으로 초기화하고, RPi5와 3-way 핸드셰이크(READY/ACK/LINK_OK)로 UART 연결을 수립한다. 이후 세션 단위로 착석 감지, 캘리브레이션 또는 측정 시작, 50 Hz 실시간 센서 수집 및 패킷 전송, 이탈(STAND) 또는 종료(STOP/QUIT) 감지를 순차적으로 반복하는 구조이며, 세션 종료 후 다시 착석 대기 상태로 복귀한다.

3.2.2. 센서 초기화

STM32는 전원 인가 시 4종 20개의 센서를 순차적으로 초기화한다. 각 센서의 설정값은 50 Hz 실시간 수집 루프에 적합하도록 결정하였으며, 주요 설정 근거는 다음과 같다. 3D ToF 2개는 최대 8×8(64존) 해상도를 지원하지만, 데이터시트[13]에 따르면 8×8 모드의 최대 측정 주파수가 15 Hz로 50 Hz 루프에서 사용이 불가능하다. 이에 4×4(16존) 모드(최대 60 Hz)를 선택하였다. SPI 클럭은 동일 데이터시트상 최대 3 MHz로 제한되어 있어, STM32의 APB2 100 MHz에서

프리스케일러(prescaler) 32(3.125 MHz)를 적용하였다[15]. 2개의 센서는 동일 SPI 버스를 공유하되 개별 NCS 핀으로 구분한다. 1D ToF 4개는 데이터시트[11]를 참조하여 타이밍 버짓(timing budget)을 20,000 μ s로 설정, 매 20 ms 사이클마다 1회 측정이 완료되도록 하였다[16]. 4개 센서가 동일 I2C 주소(0x29)를 사용하므로 TCA9548A 멀티플렉서[9]의 CH2-CH5에 배치하여 채널 전환 방식으로 접근한다.

MPU6050(관성 측정 장치(Inertial Measurement Unit, 이하 IMU), 2개)은 의사 기울기 보정용으로, 가속도계 $\pm 2g$, 디지털 저역 통과 필터(Digital Low-Pass Filter, 이하 DLPF) 20 Hz, 샘플링 50 Hz로 설정하였다[12]. I2C MUX CH6~CH7에 배치하여 1D ToF와 동일한 채널 전환 방식으로 접근한다.

HX711 12개는 데이터시트[10]의 직렬 인터페이스 규격에 따라, CLK 1개를 공유하고 DOUT 12개를 개별 GPIO에 연결하는 병렬 비트뱅잉(bit-banging) 구조로 설계하였다. 비트뱅잉 중 인터럽트(Interrupt)에 의한 타이밍 오류를 방지하기 위해 24비트 읽기 구간을 인터럽트 비활성화(_disable_irq)로 보호한다. 영점 보정(Tare)은 500 ms 안정화 후 5회 더미 읽기로 과도 응답을 제거하고, 20회 평균값을 채널별 오프셋(offset)으로 저장한다. 사용한 모듈의 RATE 핀이 PCB상 GND에 직결되어 10 Hz로 고정되어 있어, 블로킹(Blocking) 함수의 타임아웃(timeout)을 150 ms로 설정하였다.

초기화 순서는 OLED, 3D ToF, 1D ToF 센서 및 MPU6050, HX711 순으로 구성하였다. SPI와 I2C는 서로 독립된 통신 버스이므로, 한쪽 센서 초기화 실패가 다른 버스 기반 센서의 동작에 직접적인 영향을 주지 않는다. 또한 HX711은 가장 마지막에 초기화하여, 앞서 초기화된 ToF 센서들이 연속 거리 측정 모드로 이미 동작을 시작한 상태에서 로드셀의 영점 보정을 수행하도록 하였다. 이를 통해 초기화 이후 메인 루프에 진입했을 때 거리 센서는 즉시 유효한 측정값을 제공할 수 있고, 로드셀 역시 보정된 기

준값을 바탕으로 안정적으로 하중을 측정할 수 있도록 하였다.

RPi5와 UART 통신을 하기 전에는 STM32의 UART를 115,200 보드레이트(baud rate)로 설정하여 그림 18과 같이 Arduino IDE의 시리얼 모니터로 각 센서의 초기화 성공 여부를 확인하였다. 이후 RPi5와의 UART 통신을 위해 보드레이트를 921,600으로 변경하면 시리얼 모니터에서의 확인이 불가능해지므로, 그림 19와 같이 유기 발광 다이오드 디스플레이(OLED)를 상태 표시 장치로 활용하여 초기화 결과를 현장에서 확인할 수 있도록 하였다.

```
[INIT] 3D ToF...
3D[0]: OK
3D[1]: OK
[INIT] MUX + 1D ToF + MPU...
1D[0]: OK
1D[1]: OK
1D[2]: OK
1D[3]: OK
MPU[0]: OK
MPU[1]: OK
1D: 4/4, MPU: 2/2
[INIT] HX711 Tare...
HX711 offset: 645547 423181 0 0 0 0 0 0 0 0 0 0
```

그림 18. 시리얼 모니터를 이용한 STM32 센서 초기화 상태 확인 화면

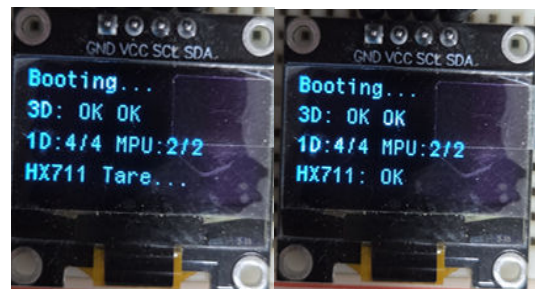


그림 19. OLED에 표시된 STM32 센서 초기화 상태 확인 화면

3.2.3. 캘리브레이션

RPi5로부터 캘리브레이션 명령(CAL)을 수신하면, 사용자가 바른 자세로 앉은 상태에서 10초간(500 프레임) 전 센서 데이터를 50 Hz로 수집하여 RPi5에 전송한다. 이때 전송되는 패킷은 측정 시의 'DAT:' 헤더 대신 'CAL:' 헤더를 사용하며, 패킷 구조와 크

기(129바이트)는 동일하다. RPi5는 수신한 500 프레임의 데이터를 평균하여 해당 사용자의 기준값(baseline)으로 저장하고, 이후 실시간 측정 시 이 기준값 대비 편차를 계산하여 자세 이상 여부를 판단한다.

캘리브레이션이 완료되면 STM32는 100 ms 대기 후 CAL_DONE 메시지를 RPi5에 전송한다. 100 ms 대기를 삽입한 이유는, 마지막 바이너리 패킷과 ASCII 메시지가 연속으로 전송될 경우 RPi5의 바이너리 수신 버퍼에 ASCII 메시지가 혼입되는 것을 방지하기 위함이다. 그림 20과 같이 캘리브레이션 진행 상황은 OLED에 실시간으로 표시된다.



그림 20. OLED에 표시된 캘리브레이션 진행 상태

3.2.4. 50 Hz 실시간 데이터 수집 및 패킷 전송

측정 루프는 20 ms(50 Hz) 주기로 동작하며, 매 사이클마다 4종의 센서를 순차적으로 폴링한 뒤 129바이트 바이너리 패킷으로 패킹하여 921,600 baud UART로 RPi5에 전송한다. 센서 읽기 순서는 통신 버스 점유 시간을 고려하여 20 ms 예산 내에서 효율적으로 운용하도록 설계하였다.

첫 번째로 3D ToF 2개를 SPI로 폴링한다. 'check_data_ready'로 새 데이터 여부를 확인하고, 준비된 경우에만 4×4(16존) 거리 데이터를 읽는다. 준비되지 않은 경우 이전 값을 유지하여 루프 지연을 방지한다.

두 번째로 1D ToF 4개를 I2C MUX 채널 전환 방식으로 읽는다. I2C 트랜잭션(transaction) 횟수를 최소화하기 위해 레지스터 직접 읽기 방식을 채택하였다[16]. 인터럽트 상태 레지스터(0x13)를 확인한 뒤, 측정 결과 레지스터(0x14)에서 12바이트를 블

록으로 읽고, 인터럽트를 클리어(0x0B)한다. 또한 RangeStatus 값을 검사하여 유효한 측정(Signal Rate Low=6, Range Valid=9, 11)일 때만 값을 갱신하고, 거리 상한선 1,200 mm를 초과하는 비정상 값은 무시하여 이전 유효값을 유지하도록 하였다.

세 번째로 MPU6050 2개의 가속도 데이터를 I2C MUX를 통해 순차적으로 읽는다. 이때 가속도 센서의 X축 가속도값(AccX)과 Z축 가속도값(AccZ)을 이용하여 'atan2' 함수를 적용하고, 그 결과를 도(degree) 단위로 변환함으로써 각 센서의 피치(pitch) 각도를 계산한다. 여기서 'atan2' 함수는 두 축 성분의 비율을 기반으로 기울기 방향과 크기를 안정적으로 산출하는 삼각함수로, 센서의 자세 변화를 각도로 표현하는 데 사용된다. 이를 통해 사용자의 등반이 또는 상체 기울기 상태를 정량적으로 판단할 수 있도록 하였다.

네 번째로 HX711 12채널 로드셀 데이터를 하이브리드 블로킹(Hybrid Blocking) 방식으로 읽는다. HX711은 10 SPS 특성으로 인해 전체 50 Hz 메인 루프보다 데이터 갱신 속도가 느리므로, 매 루프마다 새 값을 읽지 않고 약 110 ms 간격으로만 블로킹 방식의 동기 측정을 수행한다. 이때 12개 채널을 동시에 24비트 비트뱅킹 방식으로 읽고, 측정된 값에서 초기 영점 보정값을 제거하여 실제 하중값으로 사용한다. 반면 HX711의 새 데이터 읽기 시점이 아닌 나머지 루프에서는 직전에 저장된 값을 그대로 재사용하여 전체 시스템 루프 주기는 약 50 Hz로 유지한다. 즉, 센서 읽기 자체는 블로킹 방식이지만 시스템 전체 동작은 주기적으로 계속 순환하도록 구성된 하이브리드 블로킹 구조이다.

마지막으로 모든 센서 데이터가 수집되면, 전송 패킷의 무결성을 확인하기 위해 체크섬(checksum)을 계산한다. 체크섬은 전송할 데이터 바이트 전체에 대해 배타적 논리합(XOR) 연산을 순차적으로 수행하여 생성한 1바이트 검증값으로, 수신 측에서는 동일한 방식으로 다시 계산한 값과 비교하여 데이터 손상 여부를 판단할 수 있다. 본 시스템에서는 128바이트 센서 데이터에 대해 XOR 체

크섬 1바이트를 추가하여 총 129바이트 패킷을 구성하였으며, 패킷 구조는 표 7과 같다.

필드	크기 (byte)	타입	설명
header	4	char[4]	“DAT:” 또는 “CAL:”
hx711	48	int32 x12	Loadcell 1 2 채널 (offset 보정 후)
tof3d	64	uint16 x32	3D ToF 4x4존 x2개
tof1d	8	uint16 x4	1D ToF 4채널 (mm)
mpu	4	int16 x2	MPU pitch 각도 (도)
checksum	1	uint8	앞 128바이 트 XOR
합계	129		

표 7 UART 전송 데이터 패킷 구조

시리얼 모니터(115,200 baud rate)를 통해 그림 21과 같이 각 센서의 실시간 수집 데이터와 센서 수집에 소요되는 시간을 확인하였다. 실측 결과 4종 센서 수집에 약 8 ms가 소요되었다. RPi5와 UART 통신 시에는 동일 UART 핀을 921,600 baud rate로 사용하므로 시리얼 모니터를 통한 확인이 불가능하다. 이에 RPi5 연결 환경에서는 그림 22와 같이 OLED를 통해 센서 수집과 패킷 전송까지의 총 소요 시간 및 평균 루프 주파수를 확인하였다. 실측 결과 센서 수집부터 129바이트 패킷 전송 완료까지 약 9 ms가 소요되어 20 ms 예산 대비 약 55%의 여유가 확인되었으며, 평균 51 Hz로 목표 50 Hz를 안정적으로 달성하였다. 나머지 시간은 HAL_GetTick 기반으로 20 ms가 될 때까지 대기하여 정확한 50 Hz 주기를 유지한다.

```
[3D_0] 2186 30 32 26 | 32 34 38 28 | 34 1908 39 31 | 1744 1773 37 34
[3D_1] 2157 2187 2218 28 | 2001 2016 2010 26 | 28 28 1881 29 | 1724 1746 1748 28
[1D] 66 46 45 41
[MPU] 20 22
[HX] -175 -61 0 0 | 0 0 0 0 | 0 0 0 0
[TIMING] 30:5 10:2 MPU:1 HX:0 = 8 ms
```

그림 21. 시리얼 모니터 기반 실시간 센서 데이터 및 수집 시간 확인



그림 22. OLED 기반 센서 수집·전송 성능 확인 화면

3.3. RPi5 소프트웨어 개발

RPi5는 본 시스템의 핵심 분석 엔진으로, STM32로부터 수신한 센서 데이터를 실시간으로 처리하여 사용자의 자세를 분석하고 그 결과를 생성하는 역할을 수행한다.

본 소프트웨어는 Python 기반으로 구현되었으며, UART 통신 계층, 센서 데이터 보정 계층, 특징 추출 계층, 자세 판단 및 점수 계산 계층, 세션 관리 계층, 앱 통신 계층, 리포트 생성 계층으로 구성된다.

3.3.1. 센서 데이터 수신 및 패킷 파싱

RPi5는 STM32가 전송하는 129바이트 바이너리 패킷을 UART(921,600 baud)를 통해 수신한다. 패킷은 헤더, 로드셀 데이터, ToF 데이터, 기울기 데이터, 체크섬으로 구성되며, 수신부에서는 체크섬 검증을 수행하여 데이터 무결성을 확인한 뒤 각 센서 데이터로 분해한다. 이를 통해 센서 데이터 전송 과정에서 발생할 수 있는 오류를 최소화하고, 이후 분석 단계에서 안정적인 입력 데이터를 확보하도록 설계하였다.

구간	프로토콜	메시지	설명
STM32 ->RPI5	UART Binary	DAT	실시간 센서 데이터 프레임
STM32 ->RPI5	UART Binary	CAL	캘리브 레이션 데이터 프레임
RPI5-> STM32	UART ASCII	GO/ST OP	측정 시작 및 중지 제어
STM32 ->RPI	UART ASCII	SIT/ST AND	착석 및 이탈 이벤트 알림
RPI<-> App	HTTP/W ebSocket	status/ report	상태 및 리포트 전송

표 8 프로토콜

본 시스템은 STM32, RPi5, 모바일 앱 간 역할 분리를 위해 계층화된 통신 구조를 적용하였다. STM32와 RPi5 간에는 UART 기반 프로토콜을 통해 센서 데이터와 제어 명령을 송수신하고, RPi5와 앱 간에는 HTTP 및 웹소켓(WebSocket) 기반 통신을 통해 사용자 명령 처리와 실시간 상태 전송을 수행한다.

3.3.2. 센서 보정

각 센서 채널은 초기 보정값과 사전 실험을 통해 도출된 파라미터를 기반으로 처리된다. 로드셀의 경우 STM32에서 영점 보정이 선행된 값을 전송받고, RPi5에서는 채널별 질량 환산 계수를 적용하여 킬로그램 단위의 하중값으로 변환한다. 또한 매우 작은 하중값은 잡음으로 간주하여 0으로 처리함으로써 미세 변동에 의한 오검출을 줄였다. 로드셀의 보정식은 수식 4와 같다.

$$w_i = \frac{|x_i|}{c_i} \dots\dots\dots (4)$$

$$\tilde{w}_i = \begin{cases} 0, & w_i < \tau \\ w_i, & w_i \geq \tau \end{cases} \dots\dots\dots (4)$$

여기서 x_i 는 STM32에서 전송된 i 번째 로드셀 채널의 보정 후 원시값, k_i 는 해당 채널의 단위 질량당 계수, θ 는 잡음 제거를 위한 최소 하중 임계값, W_i 는 변환된 하중값을 의미한다.

한편 ToF 센서 데이터는 단순 계수 보정보다는 측정 안정화에 중점을 두어 처리하였다. 먼저 유효 거리 범위를 벗어나는 값은 무효값으로 판단하고, 이전 측정값 대비 변화량이 지나치게 큰 경우에는 급격한 점프로 간주하여 이전 안정화 값을 유지하도록 하였다. 이후 지수 이동 평균(Exponential Moving Average)을 적용하여 거리값을 평활화함으로써 순간적인 잡음과 튀는 값을 억제하였다. ToF 거리값 d 에 대한 안정화 과정은 다음과 같이 나타낼 수 있다. 먼저 유효 거리 범위를 벗어나는 값을 무효 처리하는 과정은 수식 5와 같다.

$$d_k^{(v)} = \begin{cases} d_k & d_{\min} \leq d_k \leq d_{\max} \\ \emptyset & \text{그 외의 경우} \end{cases} \dots\dots\dots (5)$$

이후 급격한 변화를 억제하고 지수 이동 평균을 적용하여 안정화하는 과정은 수식 6과 같다.

$$d_k^{(c)} = \begin{cases} \hat{d}_{k-1} & |\hat{d}_{k-1} - d_k^{(v)}| > J \\ d_k^{(v)} & |\hat{d}_{k-1} - d_k^{(v)}| \leq J \end{cases} \dots\dots\dots (6)$$

$$\hat{d}_k = \alpha d_k^{(c)} + (1 - \alpha) \hat{d}_{k-1} \dots\dots\dots (6)$$

여기서 d_k 는 현재 측정된 거리값, $d_{\min}$ 과 $d_{\max}$ 는 유효 거리 범위, J 는 허용 가능한 최대 변화량, $\hat{d}_k$ 는 이전 시점의 안정화된 거리값, α 는 지수 이동 평균 계수이다.

이와 같은 과정을 통해 센서 간 편차와 순간 잡음을 줄이고, 사용자 자세 분석의 안정성을 확보하였다.

3.3.3. 특징 추출(Feature Extraction)

보정된 센서 데이터는 자세 분석을 위한 특징값으로 변환된다. 주요 특징값으로는 등반이 좌우 압력 차이를 나타내는

'back_lr_diff', 좌판 전후 하중 이동량을 의미하는 'seat_fb_shift', 좌우 균형 상태를 나타내는 'seat_lr_diff'가 있다. 또한 ToF 센서를 기반으로 목 위치의 평균 거리 'neck_mean'과 목의 전방 이동 정도 'neck_forward_delta'를 계산하며, 척추 형태를 나타내는 'spine_curve' 지표를 통해 등받이 곡률 변화를 정량화한다.

기울기 센서를 통해 측정된 좌우 기울기 정보는 평균 기울기값과 좌우 차이값으로 가공되어 사용자의 상체 기울기를 나타내는 보조 지표로 활용된다.

이러한 특징값들은 사용자 자세 상태를 정량적으로 표현하는 핵심 지표이며, 이후 자세 판단 및 점수 계산 과정의 입력 데이터로 사용된다.

3.3.4. 자세 판단 및 상태 결정

추출된 특징값은 기준 자세 대비 변화량과 현재 센서 상태를 기반으로 규칙 기반 판단 로직에 입력된다. 시스템은 좌판 전후 하중 이동, 좌우 하중 불균형, 목 전방 이동, 상체 기울기 변화를 종합적으로 분석하여 사용자의 자세 상태를 판별한다. 구체적으로는 거북목, 상체 전방 기울기, 측면 쏠림, 걸터앉기, 다리 꼬기 의심, 뒤로 기대기, 전방 숙임, 정자세 여부를 판단하도록 설계하였다. 각 자세 상태의 판단에 사용되는 주요 임계값은 5명의 피험자를 대상으로 한 사전 실험 데이터에서 도출되었다. 피험자별로 정자세 및 7종의 나쁜 자세를 각 30초씩 유지하도록 하고, 해당 구간에서 수집된 특징값의 분포를 분석하여 자세 간 경계를 구분할 수 있는 임계값을 결정하였다.

최종적으로 사용자의 자세는 정자세, 거북목, 상체 전방 기울기, 측면 쏠림, 걸터앉기, 다리 꼬기 의심, 뒤로 기대기, 전방 숙임 상태로 구분된다. 이와 같은 다중 조건 기반 판단 구조를 통해 다양한 자세 패턴에 대해 안정적인 인식이 가능하도록 하였다.

또한 본 시스템은 향후 확장을 위해 머신러닝 기반 자세 분류 구조를 포함하고 있으나, 현재 구현 및 검증 단계에서는 규칙 기반 판단 로직이 최종 자세 상태 결정의 핵심 역할

을 수행하도록 구성하였다.

3.3.5. 자세 점수 계산

자세 점수는 시간 기반 누적 방식으로 계산되며, 나쁜 자세가 지속될수록 자세별 감점 계수에 따라 점수가 점진적으로 감소하도록 설계하였다. 특정 자세 p 가 유지되는 동안의 연속 감점은 수식 7과 같이 표현할 수 있다.

$$S_t = S_{t-1} - \lambda_p \Delta t \dots\dots\dots (7)$$

여기서 S_t 는 시점 t 에서의 자세 점수, λ_p 는 자세 p 에 대한 초당 감점 계수, Δt 는 샘플 간 시간 간격이다.

또한 나쁜 자세가 자세별 임계 지속 시간 T_p 이상 유지될 경우, 초기 경고와 함께 추가 패널티를 부여한다.

$$S_t = S_t - \beta_p \dots\dots\dots (8)$$

여기서 β_p 는 자세 p 에 대한 1차 추가 감점량이다.

이후 동일한 나쁜 자세가 일정 시간 이상 계속 유지될 경우, 재경고 단계에 따라 추가 패널티를 반복적으로 부여한다.

$$S_t = S_t - \gamma_k \dots\dots\dots (9)$$

여기서 γ_k 는 k 번째 재경고 단계에 따른 추가 감점량이다.

반대로 정자세가 유지되는 경우에는 점수가 서서히 회복되도록 하였다.

$$S_t = \min(100, S_{t-1} + \rho \Delta t) \dots\dots\dots (10)$$

여기서 ρ 는 정자세 유지 시의 초당 회복 계수를 의미한다.

이와 같은 구조를 통해 사용자의 자세 유지 습관을 정량적으로 평가하고, 나쁜 자세의 지속 시간에 따른 누적 영향을 점수에 반영할 수 있도록 하였다.

3.3.6. 세션 관리 및 데이터 저장

측정 과정은 세션 단위로 관리되며, 하나의 세션은 사용자가 착석하여 측정을 시작한 시점부터 종료 시점까지의 데이터를 포함한다.

각 세션에 대해 평균 자세 점수, 총 착석 시간, 자세별 지속 시간, 분 단위 요약 데이터가 생성된다. 또한 사용자 프로필 정보와 기준 자세값, 세션별 종합 리포트 데이터가 함께 저장되며, 모든 데이터는 SQLite 기반 데이터베이스에 기록된다. 이러한 구조를 통해 사용자의 자세 패턴에 대한 이력 관리와 후속 분석이 가능하도록 설계하였다.

3.3.7. LLM 기반 리포트 생성

측정 종료 후에는 LLM 기반 리포트 생성 모듈이 동작하여 세션 데이터를 자연어 형태로 변환한다. 본 시스템은 RPi5 환경에서 GGUF 형식의 경량 양자화 모델을 로컬로 실행하고, llama-cpp 기반 추론 엔진을 활용하여 클라우드 의존 없이 세션 종료 후 리포트를 생성하도록 설계하였다.

LLM 모델로는 Qwen3.5-0.8B[17][18]를 선정하였다. 선정 근거는 다음과 같다. 첫째, 0.8B(8억) 파라미터 규모는 RPi5의 16 GB RAM 환경에서 Q4_K_M 양자화 적용 시 약 0.6 GB의 메모리만 점유하여 분석 서버와 동시 구동이 가능하다. 둘째, 한국어 생성 품질이 동급 경량 모델 대비 우수하여 자연스러운 리포트 생성이 가능하다[19]. 셋째, RPi5에서 llama-cpp 기반 추론 시 약 15~25초 내에 리포트 생성이 완료되어 세션 종료 후 사용자 대기 시간이 허용 범위 내에 있다.

LLM의 입력 데이터는 전체 세션 요약 정보와 시간 흐름 기반 분 단위 요약 데이터로 구성되며, 이를 기반으로 사용자의 자세 패턴을 분석한다. 모델은 분석 결과를 바탕으로 전체 자세 상태 요약, 시간에 따른 자세 변화 분석, 맞춤형 운동 추천, 생활 습관 개선 조언을 포함하는 구조화된 리포트를 생성한다. 생성된 리포트는 단순 통계 정보를 넘어 사용자의 자세 문제 원인과 개선 방향을 직관적으로 이해할 수 있도록 구성된다.

또한 추론 실패 시에도 동일한 출력 구조를 유지할 수 있도록 규칙 기반 대체 리포트 생성 경로를 함께 구성하여 시스템의 안정성을 확보하였다.

3.3.8. 앱 통신 및 실시간 데이터 전송

RPi5는 FastAPI 기반 서버를 통해 모바일 앱과 통신하며, 사용자 명령은 HTTP POST 방식으로 수신하고 분석 결과는 웹소켓을 통해 실시간으로 전송한다.

전송되는 데이터는 실시간 자세 상태를 나타내는 'realtime_status', 센서 분포를 시각화하기 위한 'sensor_distribution', 분 단위 요약 데이터인 'minute_summary', 세션 전체 요약인 'overall_summary', 그리고 LLM 기반 종합 리포트인 'enhanced_report'로 구성된다.

특히 'sensor_distribution' 데이터는 의자의 실제 센서 배치를 반영하여 앱에서 직관적인 시각화가 가능하도록 설계하였으며, 이를 통해 사용자는 자신의 자세 상태를 실시간으로 쉽게 인지할 수 있다.

3.4. 모바일 앱 개발

3.4.1. 통신 프로토콜 설계

RPi5 서버(http://<IP>:8000)와의 JSON 기반 HTTP 통신 명세를 먼저 확정하였다. 앱에서 RPi5로의 명령 전송은 HTTP POST /command 단일 endpoint를 통해 수행되며, 총 8종의 command를 JSON 형식으로 전송한다.

기능	command
프로필 등록	submit_profile
프로필 선택	select_profile
캘리브레이션 시작	start_calibration
캘리브레이션 생략	skip_calibration
측정 시작	start_measurement
측정 중지	stop_measurement
재캘리브레이션 요청	request_recibration
자리 이탈 후 재개/거부	resume_after_stand, decline_resume_after_stand

표 9 RPi5 통신 명령어

RPi5에서 앱으로의 데이터 전달은 HTTP

Polling 방식으로 3개 endpoint를 각각 다른 주기로 조회한다.

Endpoint	주기	용도
GET /meta	0.8초	시스템 단계(stage) 변화 감지, UI 화면 전환 트리거
GET /status	0.5초	실시간 자세 데이터(점수, posture 플래그, 센서별 수치)
GET /report	2초	분 단위 요약 및 세션 종합 리포트

표 10 데이터 통신 프로토콜 및 요청 빈도

RPi5 응답 데이터는 meta, realtime_status, minute_summary, overall_summary, stand_event의 5종으로 구성된다. RPi5는 command 수신 시 항상 JSON ack를 반환하며, 성공 시 { "ok": true, "message": "command_received" }, 실패 시 { "ok": false, "message": "invalid_command" } 형식이다.

3.4.2. Flutter 프로젝트 구조 설계

개발 언어 Dart[6], 프레임워크 Flutter(Material3)[3], 상태 관리 Provider 패턴[20]으로 결정하였다.

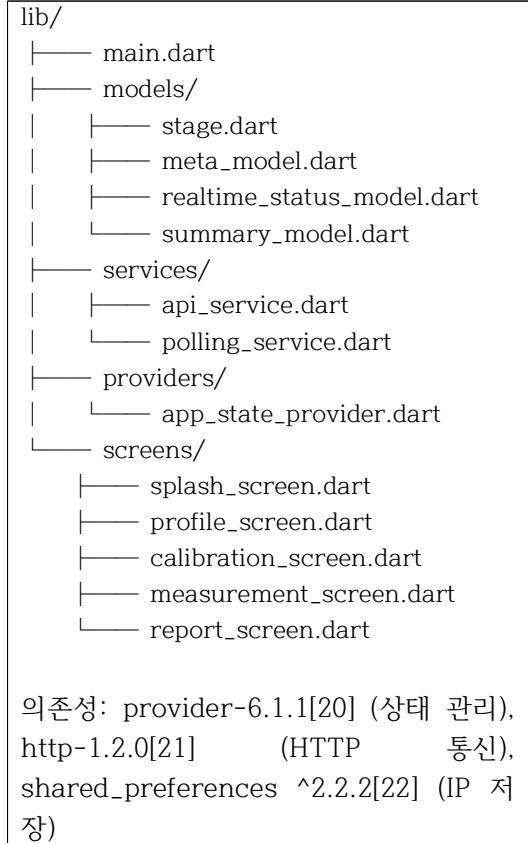


표 11. 앱 디렉터리 구조

3.4.3. 시스템 단계 기반 화면 전환 구현

RPi5가 관리하는 17개 스테이지를 기준으로 5개 화면에 자동 라우팅(Routing)되는 구조를 구현하였다.

시스템 단계 범위	→	화면
연결전/연결실패	→	SplashScreen (IP 입력)
uart_link_ready ~ profile_loaded	→	ProfileScreen
wait_calibration_decision ~ calibration_completed	→	CalibrationScreen
wait_start_decision ~ measuring ~ paused	→	MeasurementScreen
session_saved ~ session_ended	→	ReportScreen

표 12 시스템 단계에 따른 화면 출력

앱의 화면 전환은 사용자의 버튼 입력이 아닌 RPi5 서버의 상태 변화에 의해 결정된다. 일반적인 모바일 앱에서는 버튼을 누르면 앱이 직접 다음 화면으로 이동하는 방식을 사용하지만, 본 시스템에서는 앱이 RPi5 서버에 명령을 전송한 후 곧바로 화면을 전환하지 않는다. RPi5 서버가 해당 명령을 처리하고 현재 시스템 단계를 웹소켓으로 앱에 전송하면, 앱의 전역 상태 관리 객체가 이를 수신하여 내부 상태를 갱신한다. 최상위 화면 라우터는 이 상태를 실시간으로 감시하고 있으며, 시스템 단계 값이 변경되는 즉시 해당 단계에 대응하는 화면을 자동으로 표시한다.

예를 들어, 사용자가 캘리브레이션 시작 버튼을 누르면 앱은 RPi5에 명령만 전송하고, RPi5가 캘리브레이션을 개시하며 시스템 단계를 'calibrating'으로 변경하여 전송한 시점에 비로소 캘리브레이션 화면이 나타난다. 이러한 구조를 통해 앱과 서버의 상태가 항상 동기화되며, 통신 지연이나 명령 처리 실패 시에도 앱이 서버와 다른 화면을 표시하는 문제를 방지할 수 있다.

3.4.4. 통신 계층 구현

(1) ApiService

RPi5 base URL 설정, 'GET /meta', 'GET /status', 'GET /report', 'POST /command' 4개 메서드를 구현하였다.

(2) PollingService

Timer.periodic으로 3개 endpoint를 각 다른 주기로 폴링한다.

- /meta : 0.8초 주기
- /status : 0.5초 주기
- /report : 2초 주기

콜백 방식으로 AppStateProvider에 데이터를 전달한다.

(3) AppStateProvider

Polling 콜백에서 시스템 단계 변경 감지, 실시간 자세 업데이트, stand event 처리(중복 방지 플래그),

분 요약 누적, 세션 결과 수신, AI 분석 결과 수신 시 모바일 앱 저장하여 UI에서 즉시 반영되도록 처리한다.

3.4.5. 화면별 UI 구현

(1) SplashScreen

RPi5 IP 주소 입력, 최근 보고서 5개까지 확인 할 수 있다.

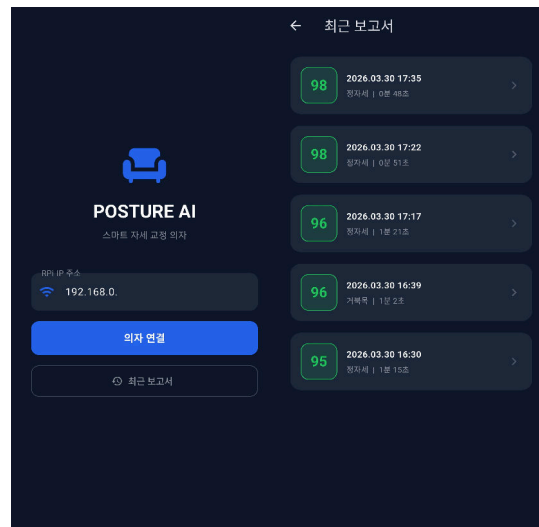


그림 23. 앱 시작화면(좌), 최근 보고서 목록(우)

(2) ProfileScreen

신규 프로필 등록 (이름, 키, 체중, 작업/휴식 시간) 및 기존 프로필 선택. submit_profile/select_profile command를 POST /command로 전송한다.

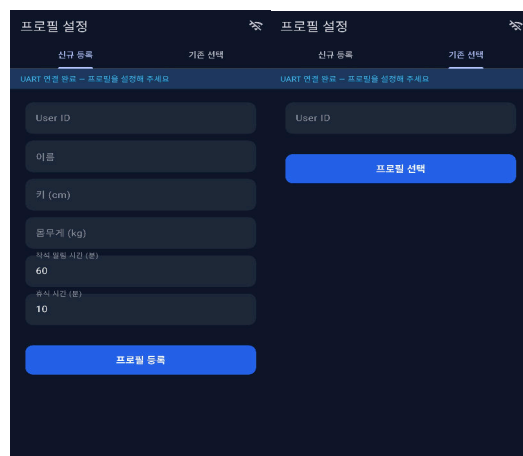


그림 24. 신규 프로필 등록(좌), 기존 프로필 접속(우)

(3) CalibrationScreen

캘리브레이션 시작(start_calibration) 또는 생략(skip_calibration) 선택. 진행 상태를 화면에 표시한다.

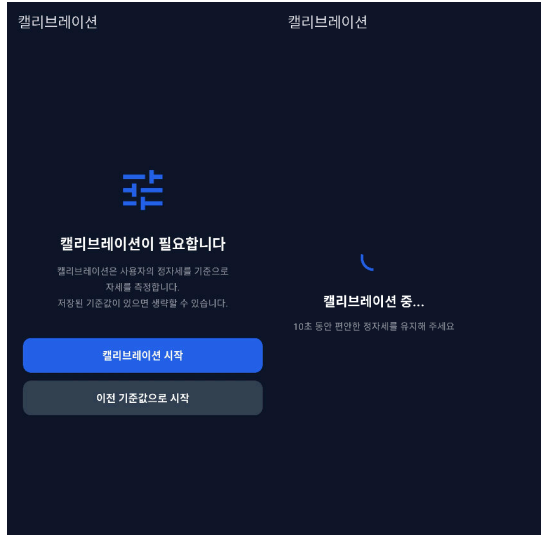


그림 25. 캘리브레이션 선택 화면(좌), 캘리브레이션 진행 화면(우)

(4) MeasurementScreen (핵심 화면)

측정 화면은 3개 탭으로 구성하였다. 대시보드 탭에서는 실시간 자세 점수, 감지된 자세 유형 표시, 좌우 균형·척추 거리·목 거리의 집계 모니터링 지표를 제공한다. 압력 분포 탭에서는 의자 형태를 모방한 시각화 위젯을 통해 등받이 영역의 좌우 압력 8개와 중앙 척추 거리 센서 4개를 3열 4행 격자로, 좌석 영역의 압력 4개를 2행 2열 격자로 표시하며, 각 셀의 배경색이 양호·주의·위험 상태에 따라 실시간으로 변화한다. 각 센서 수치에 따라 초록에서 노랑, 빨강으로 색상이 변화하여 자세 상태를 직관적으로 표시하도록 구현하였다. 프로토콜 탭에서는 현재 시스템 단계, 연결 상태, polling 주기 등 통신 상태를 모니터링할 수 있으며, 로그 탭에서는 시스템 단계 변경, 자세 변경, 경고 이벤트 등의 실시간 통신 로그를 누적하여 표시한다. 자리 이탈(stand_event) 감지 시에는 다이얼로그를 통해 재개 또는 종료를 선택할 수 있도록 구현하였다.

(5) ReportScreen

세션 종합 결과 표시 단계이다. 데이터 기반 점수, 자세 분포, 측정 시간, AI 분석 코멘트 등을 포함한다.

RPi5에서 Qwen3.5-0.8B 모델을 로컬 추론하여 자세 분석 피드백을 생성하고, 앱에서 이를 수신·표시하는 구조를 구현하였다.

RPi5로부터 수신한 LLM 분석 결과 (자세 교정 제안, 세션 요약 코멘트 등)를 표시한다. 기존 Polling 구조 (GET /status 또는 GET /report)의 응답 JSON에 AI 분석 필드가 포함되어 별도 endpoint 추가 없이 수신된다.



그림 26. 대쉬보드 위젯(좌), 앱 센서 위젯(우)



그림 27. 자리아탈 감지(좌), LLM기반 분석 리포트(우)

3.4.6. RPi5 연동 테스트 및 디버깅

실제 RPi5 서버와 연결하여 전체 흐름을 통합 테스트하였다. 연결, 프로필 등록 및 선택, 캘리브레이션, 실시간 측정, AI 분석 수신, 세션 리포트 순으로 동작을 검증하였다. 시스템 단계 전이 로직과 데이터 형식이 실제 환경에서 정상 동작하는지 확인하였다. LLM 분석 결과의 수신 지연 및 JSON 파싱 안정성도 함께 검증하였다.

3.5. 최종 테스트

시스템의 전체 동작을 검증하기 위해 최종 통합 테스트를 수행하였다. 테스트는 하드웨어 안정성, 자세 판별 정확도, LLM 리포트 품질의 3개 항목으로 구성하였다.

하드웨어 안정성 테스트에서는 30분 이상의 연속 측정 세션을 3회 반복하여 센서 데이터 수집의 연속성과 UART 패킷 손실률을 확인하였다. 체크섬 검증을 통해 확인한 패킷 손실률은 0.1% 미만으로, 50 Hz 실시간 수집 루프가 안정적으로 동작함을 검증하였다.

자세 판별 정확도 테스트에서는 5명의 피험자를 대상으로 8종의 자세(정자세, 거북목, 상체 전방 기울기, 측면 쏠림, 걸터앉기, 다리 꼬기 의심, 뒤로 기대기, 전방 숙임)를 각 30초씩 유지하도록 하고, 시스템의 판별 결

과와 실제 자세를 비교하였다. 전체 평균 판별 정확도는 약 85%로, 정자세와 뒤로 기대기는 90% 이상의 높은 정확도를 보였으나, 다리 꼬기 의심과 걸터앉기는 상대적으로 낮은 정확도를 나타내어 향후 임계값 조정 또는 머신러닝 기반 판별 로직 도입이 필요한 것으로 확인되었다.

LLM 리포트 품질 테스트에서는 Qwen3.5-0.8B 모델이 RPi5에서 평균 약 20초 내에 리포트 생성을 완료하였으며, 생성된 리포트가 세션 데이터의 주요 특성을 반영하여 구조화된 피드백을 제공하는지 확인하였다. 대부분의 경우 자세 요약과 운동 추천이 적절하게 생성되었으나, 0.8B 경량 모델의 한계로 인해 일부 세션에서 반복적인 표현이 관찰되었다. 추론 실패 시에는 규칙 기반 대체 리포트가 정상적으로 생성되어 시스템 안정성이 확보됨을 확인하였다.



그림 28. 새우 자세

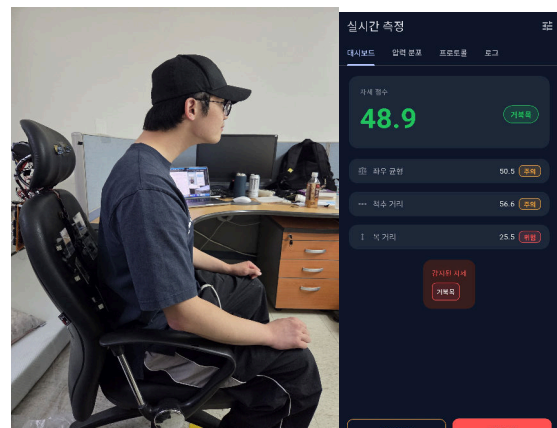


그림 29. 거북목 자세

4. 사용한 제품 리스트

제품명	개수	디바이스마트 상품번호
라즈베리파이5 (Raspberry Pi 5) 16GB + 가이드북	1	15666226
HX711 Loadcell 측정 24비트 AD 컨버터 모듈	12	1327440
SATEL-VL53L8	2	15575544
MPU-6050 6축 자이로 가속도 센서 모듈 GY-521	2	1247052
pyboard STM32F411CEU6 C타입 소형 개발 보드	1	15028296
Load Cell 50kg 무게측정 센서	8	-
4선식 Load Cell 20kg 무게측정 센서	8	-
LM2596HV 강하형 DC-DC 가변 컨버터	1	1321158
XL4005 DC-DC 5A 강하형 스텝다운 모듈	2	15960407
VARCS 4S Lipo 배터리 130C 4셀8400mAh	1	-
TCA9548A I2C Mulitplexer Module	1	-

표 13. 사용한 제품 리스트

5. 기타

5.1. 소스코드

별도의 파일에 첨부함.

6. 참고문헌

1. *STMicroelectronics*, "STM32F411xC/xE – ARM Cortex-M4 32-bit MCU," Datasheet (DS10314), <https://www.st.com/resource/en/datasheet/stm32f411ce.pdf>
2. *Raspberry Pi Foundation*, "Raspberry Pi 5 Documentation,"

- <https://www.raspberrypi.com/documentation/computers/raspberry-pi-5.html>
3. *Google*, "Flutter Documentation," <https://docs.flutter.dev/>
 4. *STMicroelectronics*, "STM32CubeIDE – Integrated Development Environment for STM32," <https://www.st.com/en/development-tools/stm32cubeide.html>
 5. *STMicroelectronics*, "STM32CubeMX – STM32 configuration and code generation tool," <https://www.st.com/en/development-tools/stm32cubemx.html>
 6. *Google*, "Dart Programming Language Documentation," <https://dart.dev/guides>
 7. *WeAct Studio*, "WeAct STM32F411CEU6 (BlackPill V3.1)," <https://github.com/WeActStudio/WeActStudio.MiniSTM32F4x1>
 8. *STMicroelectronics*, "STM32F411xC/xE Reference Manual," Reference Manual (RM0383), https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xc-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
 9. *Texas Instruments*, "TCA9548A – Low-Voltage 8-Channel I2C Switch with Reset," Datasheet (SCPS207H), <https://www.ti.com/lit/ds/symlink/tca9548a.pdf>
 10. *AVIA Semiconductor*, "HX711 – 24-Bit Analog-to-Digital Converter for Weigh Scales," Datasheet, https://cdn.sparkfun.com/datasheets/Sensors/ForceFlex/hx711_english.pdf
 11. *STMicroelectronics*, "VL53L0X – World's smallest Time-of-Flight ranging and gesture detection sensor," Datasheet (DS11555), <https://www.st.com/resource/en/datasheet/vl53l0x.pdf>
 12. *InvenSense (TDK)*, "MPU-6000 and MPU-6050 Product Specification," Revision 3.4, Document Number PS-MPU-6000A-00, <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>
 13. *STMicroelectronics*, "VL53L8CX – Time-of-Flight 8x8 multizone ranging sensor with wide field of view," Datasheet (DS14161),

- <https://www.st.com/resource/en/datasheet/vl53l8cx.pdf>
14. *STMicroelectronics*, "SATEL-VL53L8 – Breakout board for VL53L8CX," <https://www.st.com/en/evaluation-tools/satel-vl53l8.html>
 15. *STMicroelectronics*, "VL53L8CX Ultra Lite Driver (ULD) User Manual," User Manual (U M 3 1 0 9) , https://www.st.com/resource/en/user_manual/um3109-a-guide-to-using-the-vl53l8cx-ultra-lite-driver-uld-stmicroelectronics.pdf
 16. *STMicroelectronics*, "VL53L0X API User Manual," User Manual (UM2039), https://www.st.com/resource/en/user_manual/um2039-world-smallest-time-of-flight-ranging-and-gesture-detection-sensor-application-programming-interface-stmicroelectronics.pdf
 17. *Qwen Team, Alibaba Cloud*, "Qwen3.5 – GitHub Repository," <https://github.com/QwenLM/Qwen3.5>
 18. *Hugging Face*, "Qwen/Qwen3.5-0.8B – Model Card," <https://huggingface.co/Qwen/Qwen3.5-0.8B>
 19. *Qwen Team, Alibaba Cloud*, "Qwen3.5 Research," <https://qwen.ai/research>
 20. *Remi Rousselet*, "provider – A wrapper around InheritedWidget," Pub.dev, <https://pub.dev/packages/provider>
 21. *Dart Team*, "http – A composable, Future-based library for making HTTP requests," Pub.dev, <https://pub.dev/packages/http>
 22. *Flutter Team*, "shared_preferences – Wraps platform-specific persistent storage," Pub.dev, https://pub.dev/packages/shared_preferences